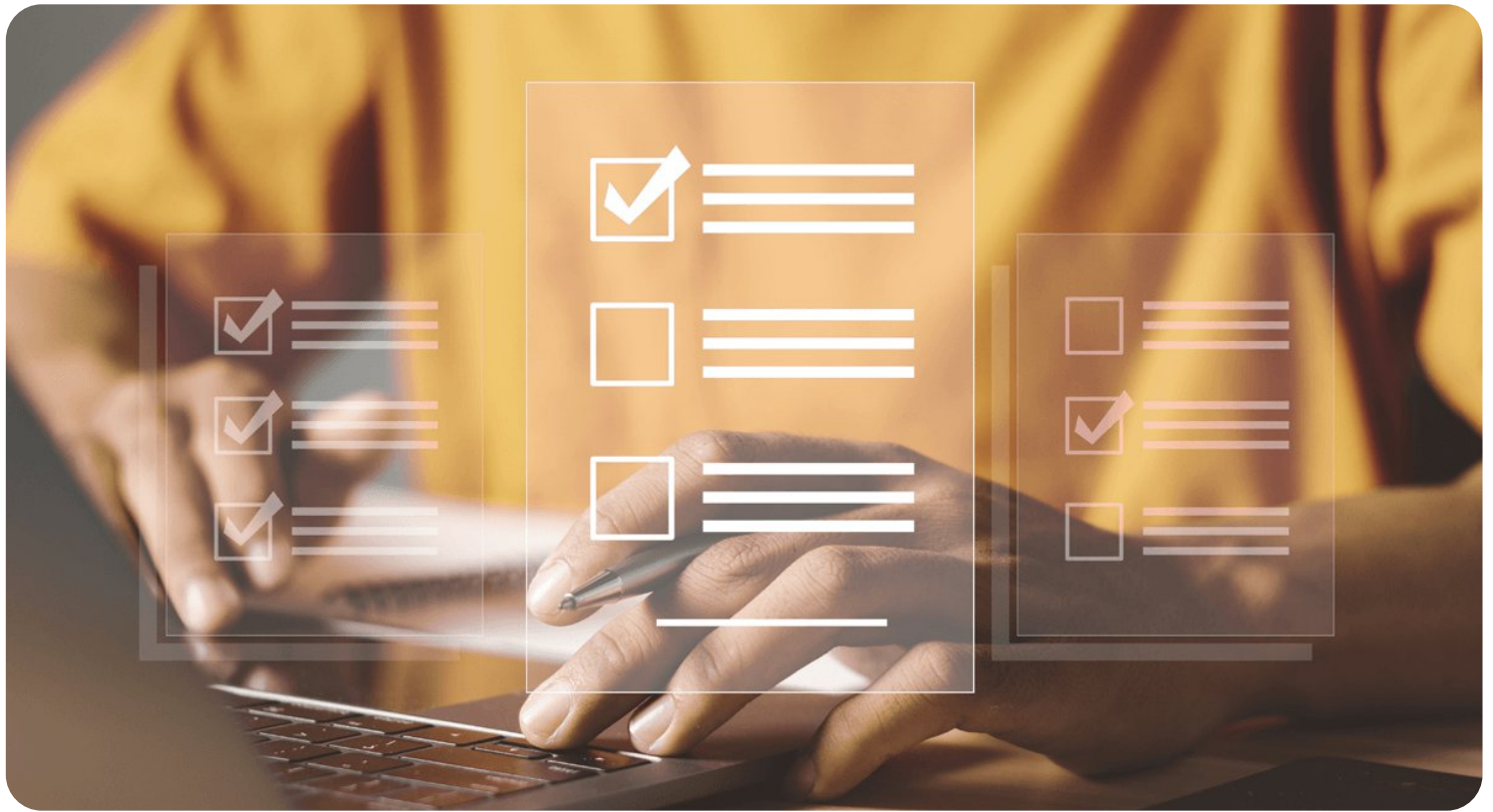




Semana 9

Desarrollo Cloud Native (CDY2204)

Informe escrito

Formato de respuesta

Nombre estudiante:	Natalia Alvarado Egor Llancaichun
Asignatura: Desarrollo Cloud Native	Carrera: Analista Programador
Profesor: Alonso Castillo	Fecha: 19-07-2026

Índice

1. Introducción
2. Arquitectura general de la solución
3. Configuración de Oracle Autonomous Database
4. Configuración de Amazon EC2
5. Configuración de Amazon EFS
6. Configuración de Amazon S3
7. Configuración de RabbitMQ
8. Configuración del microservicio productor
9. Configuración del microservicio consumidor
10. Configuración de Azure AD B2C
11. Configuración de Amazon API Gateway
12. Configuración de Docker Compose
13. Configuración de variables de entorno y secretos
14. Repositorio de GitHub
15. Conclusiones
16. Repositorio

1. Introducción

El presente documento describe paso a paso el proceso de configuración de los servicios cloud utilizados en el sistema **Gestión de Guías de Despacho**.

La solución fue diseñada utilizando una arquitectura basada en microservicios, separando las responsabilidades de publicación y consumo de mensajes en dos aplicaciones Spring Boot independientes. El microservicio productor recibe las solicitudes relacionadas con las guías de despacho y publica los mensajes en RabbitMQ, mientras que el microservicio consumidor procesa dichos mensajes y registra los resultados en Oracle Autonomous Database.

La infraestructura implementada integra servicios de Amazon Web Services, Oracle Cloud y Microsoft Azure. Entre ellos se encuentran Amazon EC2, Amazon API Gateway, Amazon EFS, Amazon S3, Oracle Autonomous Database, Azure AD B2C y RabbitMQ.

El objetivo de este documento es explicar únicamente la configuración de los servicios utilizados. Las pruebas de funcionamiento, respuestas obtenidas mediante Postman, procesamiento de mensajes, registros en Oracle y validación de la cola de errores se presentan de forma separada en el video de evidencia.

2. Arquitectura general de la solución

La solución utiliza una arquitectura distribuida compuesta por dos microservicios desplegados en contenedores Docker dentro de una instancia Amazon EC2.

El microservicio productor se ejecuta en el puerto 8081 y se encarga de recibir las solicitudes HTTP, gestionar las guías de despacho, almacenar archivos y publicar mensajes en RabbitMQ.

El microservicio consumidor se ejecuta en el puerto 8082 y se encarga de recibir los mensajes publicados en la cola, validar la información, registrar los resultados en Oracle y administrar el flujo de errores.

Amazon API Gateway funciona como punto único de acceso público. Las rutas protegidas requieren un token JWT emitido por Azure AD B2C.

RabbitMQ permite desacoplar la recepción de solicitudes del procesamiento interno. Oracle Autonomous Database almacena las guías y sus resultados de procesamiento. Amazon EFS proporciona almacenamiento compartido en EC2, mientras que Amazon S3 conserva los archivos generados de forma persistente.

Componentes utilizados

Componente	Función
Amazon API Gateway	Expone las rutas públicas de la solución.
Azure AD B2C	Emite y valida los tokens JWT.
Amazon EC2	Hospeda los contenedores de la aplicación.
Microservicio productor	Gestiona las solicitudes y publica mensajes.
RabbitMQ	Administra las colas y el procesamiento asíncrono.
Microservicio consumidor	Consume y procesa los mensajes.
Oracle Autonomous Database	Almacena las guías y los resultados.
Amazon EFS	Mantiene archivos compartidos dentro de EC2.
Amazon S3	Almacena los documentos de manera persistente.
Docker Compose	Orquesta los servicios desplegados.
GitHub	Mantiene el código fuente y el control de versiones.

3. Configuración de Oracle Autonomous Database

Oracle Autonomous Database fue utilizada como servicio de persistencia para almacenar las guías de despacho, los resultados procesados y los registros de error.

3.1 Creación o selección de la base de datos

Para comenzar, se ingresó al portal de Oracle Cloud y se seleccionó el servicio **Autonomous Database**.

Dentro del servicio se creó o seleccionó la base de datos utilizada por el proyecto. Posteriormente se verificó que su estado se encontrara disponible.

Los principales datos requeridos para la conexión fueron:

- Nombre de la base de datos.
- Usuario de conexión.
- Contraseña.
- Nombre del servicio TNS.
- Wallet de conexión.
- Región configurada.

3.2 Descarga del Wallet

Desde la base de datos autónoma se ingresó a la opción:

Conexión de base de datos

Posteriormente se seleccionó:

Descargar Wallet

Oracle solicitó definir una contraseña para proteger el archivo descargado. El archivo comprimido contiene los certificados y archivos de configuración necesarios para realizar una conexión mTLS.

Entre los archivos más importantes se encuentran:

tnsnames.ora

sqlnet.ora

cwallet.sso

ewallet.p12

ojdbc.properties

El Wallet fue extraído en un directorio protegido y no fue incorporado al repositorio GitHub.

3.3 Creación de las tablas

Una vez configurada la conexión, se utilizó Oracle SQL Developer o DBeaver para ejecutar el script de creación de tablas.

Las tablas principales utilizadas fueron:

GUIAS_DESPACHO

GUIAS_DESPACHO_MQ

La tabla GUIAS_DESPACHO almacena la información principal de cada guía.

La tabla GUIAS_DESPACHO_MQ conserva la información relacionada con el procesamiento asíncrono, incluyendo estado, resultado y detalle de errores.

El script fue ejecutado utilizando el usuario configurado en Oracle Autonomous Database.

3.4 Copia del Wallet hacia EC2

El Wallet fue copiado hacia la instancia EC2 utilizando el comando scp.

Ejemplo de comando:

```
scp -i "OracleCloud123.pem" \  
    -r "Wallet_Semana3" \  
    ubuntu@IP_PUBLICA_EC2:/home/ubuntu/wallet
```

Posteriormente se verificó que los archivos se encontraran disponibles dentro del servidor:

```
ls -la /home/ubuntu/wallet
```

3.5 Montaje del Wallet en Docker

Dentro del archivo docker-compose.prod.yml, el directorio del Wallet fue montado en ambos microservicios.

Ejemplo:

volumes:

```
- /home/ubuntu/wallet:/wallet:ro
```

El modificador :ro permite que los contenedores utilicen el Wallet en modo de solo lectura.

La variable utilizada por el driver de Oracle fue:

```
TNS_ADMIN=/wallet
```

De esta forma, ambos microservicios pueden acceder a Oracle utilizando los certificados almacenados dentro del contenedor.

4. Configuración de Amazon EC2

Amazon EC2 fue utilizado para hospedar los contenedores Docker correspondientes al productor, consumidor y RabbitMQ.

4.1 Creación de la instancia

Desde la consola de AWS se ingresó a:

EC2

→ Instancias

→ Lanzar instancia

Se configuraron los siguientes elementos:

- Nombre de la instancia.
- Imagen Ubuntu Server.
- Tipo de instancia.
- Par de claves.
- Red virtual.
- Subred.
- Dirección IP pública.
- Grupo de seguridad.
- Almacenamiento.

La instancia fue configurada con una llave privada en formato .pem, necesaria para realizar la conexión SSH.

4.2 Configuración del grupo de seguridad

En el grupo de seguridad asociado a EC2 se configuraron las reglas de entrada necesarias.

Puerto Protocolo		Función
22	TCP	Acceso SSH.
8081	TCP	Microservicio productor.
8082	TCP	Microservicio consumidor.
5672	TCP	Comunicación AMQP con RabbitMQ.
15672	TCP	Panel de administración de RabbitMQ.
2049	TCP	Comunicación NFS con Amazon EFS.

Para mejorar la seguridad, el puerto 22 debe limitarse únicamente a las direcciones IP autorizadas.

Los puertos de administración tampoco deben quedar abiertos globalmente en un entorno productivo.

4.3 Conexión por SSH

La conexión se realizó utilizando el siguiente comando:

```
ssh -i "OracleCloud123.pem" ubuntu@IP_PUBLICA_EC2
```

Antes de utilizar la llave se configuraron permisos restringidos:

```
chmod 400 OracleCloud123.pem
```

4.4 Instalación de herramientas

Dentro de la instancia se actualizó el sistema:

```
sudo apt update
```

```
sudo apt upgrade -y
```

Posteriormente se instalaron Git y Docker:

```
sudo apt install git -y
```

```
sudo apt install docker.io docker-compose-plugin -y
```

El usuario fue agregado al grupo Docker:

```
sudo usermod -aG docker ubuntu
```

Después de cerrar y volver a iniciar sesión, se verificó la instalación:

```
docker --version
```

```
docker compose version
```

```
git --version
```

4.5 Clonación del repositorio

El repositorio se clonó dentro del servidor:

```
git clone https://github.com/LadyRed145/GestionGuiasDespacho_Grupo13.git
```

Posteriormente se ingresó al proyecto:

```
cd GestionGuiasDespacho_Grupo13
```

Para actualizar el código se utilizó:

```
git pull origin main
```

5. Configuración de Amazon EFS

Amazon EFS fue utilizado como sistema de archivos compartido para mantener archivos accesibles desde los contenedores ejecutados dentro de EC2.

5.1 Creación del sistema de archivos

Desde la consola de AWS se ingresó a:

EFS

→ Crear sistema de archivos

Se seleccionó la misma VPC utilizada por la instancia EC2.

Posteriormente se configuraron los puntos de montaje en las subredes correspondientes.

5.2 Configuración del grupo de seguridad

El grupo de seguridad asociado a EFS debe permitir tráfico NFS mediante el puerto:

2049 TCP

Como origen se utilizó el grupo de seguridad de la instancia EC2.

Esto permite que solamente la instancia autorizada pueda montar el sistema de archivos.

5.3 Instalación del cliente NFS

Dentro de EC2 se instaló el cliente requerido:

```
sudo apt install nfs-common -y
```

Luego se creó el directorio de montaje:

```
sudo mkdir -p /mnt/efs/guias
```

5.4 Montaje de EFS

El sistema EFS fue montado utilizando el DNS entregado por AWS.

Ejemplo:

```
sudo mount -t nfs4 \
-o nfsvers=4.1 \
fs-IDENTIFICADOR.efs.us-east-1.amazonaws.com:/ \
/mnt/efs/guias
```

Se verificó el montaje:

```
df -h
```

Para que el montaje permanezca disponible después de reiniciar EC2, se puede registrar en `/etc/fstab`.

Ejemplo:

```
fs-IDENTIFICADOR.efs.us-east-1.amazonaws.com:/ /mnt/efs/guias nfs4 defaults,_netdev 0 0
```

5.5 Integración con Docker

Dentro de Docker Compose se montó el directorio EFS:

volumes:

```
- /mnt/efs/guias:/app/archivos
```

De esta forma, los archivos generados dentro de los contenedores permanecen almacenados en EFS aunque los servicios sean recreados.

6. Configuración de Amazon S3

Amazon S3 fue utilizado para almacenar de forma persistente los archivos asociados a las guías de despacho.

6.1 Creación del bucket

Desde AWS se ingresó a:

Amazon S3

→ Crear bucket

Se configuraron los siguientes elementos:

- Nombre único del bucket.
- Región.
- Propiedad de objetos.
- Bloqueo de acceso público.
- Versionado, cuando corresponde.
- Cifrado predeterminado.

El acceso público se mantuvo bloqueado debido a que los archivos son administrados directamente por la aplicación.

6.2 Organización mediante prefijos

Los objetos se almacenan utilizando prefijos que permiten mantenerlos organizados.

Ejemplo:

2026-07-19/Logistica_Andina_Express/guia-GD-801.txt

Esta estructura permite identificar la fecha, transportista y número de guía.

6.3 Configuración de permisos IAM

Se creó o utilizó una identidad IAM con permisos limitados al bucket correspondiente.

Los permisos requeridos son:

s3:PutObject

s3:GetObject

s3:ListBucket

s3:DeleteObject

La política debe restringirse al bucket utilizado por el proyecto.

Ejemplo de estructura:

```
{
```

```

"Version": "2012-10-17",
"Statement": [
  {
    "Effect": "Allow",
    "Action": [
      "s3:PutObject",
      "s3:GetObject",
      "s3:DeleteObject"
    ],
    "Resource": "arn:aws:s3:::NOMBRE_BUCKET/*"
  },
  {
    "Effect": "Allow",
    "Action": "s3:ListBucket",
    "Resource": "arn:aws:s3:::NOMBRE_BUCKET"
  }
]
}

```

6.4 Variables utilizadas

Las credenciales fueron configuradas mediante variables de entorno:

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

AWS_REGION

AWS_S3_BUCKET

Los valores reales no fueron incorporados al repositorio.

El archivo .env.docker fue protegido mediante permisos restrictivos:

```
chmod 600 .env.docker
```

7. Configuración de RabbitMQ

RabbitMQ fue utilizado como broker de mensajería para desacoplar la publicación de las guías de su procesamiento.

7.1 Incorporación de RabbitMQ mediante Docker

Dentro de Docker Compose se utilizó la imagen:

rabbitmq:3-management

Ejemplo de configuración:

rabbitmq:

image: rabbitmq:3-management

container_name: gestion-guias-rabbitmq

ports:

- "5672:5672"

- "15672:15672"

environment:

RABBITMQ_DEFAULT_USER: \${RABBITMQ_USERNAME}

RABBITMQ_DEFAULT_PASS: \${RABBITMQ_PASSWORD}

healthcheck:

test: ["CMD", "rabbitmq-diagnostics", "ping"]

interval: 10s

timeout: 5s

retries: 10

El puerto 5672 se utiliza para la comunicación AMQP.

7.2 Configuración de la cola principal

La cola principal recibe los mensajes publicados por el productor.

Nombre utilizado:

`guias.despacho.queue`

También se configuró un exchange y una routing key para dirigir los mensajes hacia la cola correspondiente.

Ejemplo conceptual:

Exchange: `guias.despacho.exchange`

Routing key: `guias.despacho.routing.key`

Queue: `guias.despacho.queue`

7.3 Configuración de la cola de errores

Para mantener la trazabilidad de los mensajes con problemas se configuró una cola independiente:

`guias.despacho.error.queue`

Ejemplo conceptual:

Exchange: `guias.despacho.error.exchange`

Routing key: `guias.despacho.error.routing.key`

Queue: `guias.despacho.error.queue`

La cola de errores permite almacenar los mensajes que no pueden ser procesados correctamente.

7.4 Configuración de exchanges y bindings

En la aplicación Spring Boot se definieron los beans correspondientes a:

- Cola principal.
- Cola de errores.
- Exchange principal.
- Exchange de errores.
- Binding principal.
- Binding de errores.

El binding relaciona cada cola con su exchange mediante una routing key.

7.5 Variables de conexión

Los dos microservicios utilizan las siguientes variables:

RABBITMQ_HOST

RABBITMQ_PORT

RABBITMQ_USERNAME

RABBITMQ_PASSWORD

Dentro de Docker Compose, el host corresponde al nombre del servicio:

rabbitmq

De esta manera, los contenedores se comunican utilizando la red interna de Docker.

8. Configuración del microservicio productor

El microservicio productor fue implementado como una aplicación Spring Boot independiente.

8.1 Puerto de ejecución

El servicio fue configurado en el puerto:

8081

Configuración:

server.port=8081

8.2 Responsabilidades

El productor se encarga de:

- Recibir solicitudes HTTP.
- Crear guías de despacho.
- Consultar guías.
- Actualizar registros.
- Eliminar registros.
- Generar y almacenar archivos.
- Publicar mensajes en RabbitMQ.
- Exponer el health check del productor.

8.3 Conexiones configuradas

El productor utiliza:

- Oracle Autonomous Database.
- RabbitMQ.
- Amazon S3.
- Amazon EFS.
- Azure AD B2C.
- Amazon API Gateway.

8.4 Health check

Se configuró una ruta específica:

GET /api/productor/health

También se habilitó Spring Boot Actuator:

GET /actuator/health

En la configuración se habilitaron los detalles necesarios para verificar las dependencias de la aplicación.

Ejemplo:

management.endpoints.web.exposure.include=health,info

management.endpoint.health.show-details=always

8.5 Dockerfile del productor

El Dockerfile compila y ejecuta el archivo JAR.

Ejemplo:

FROM eclipse-temurin:21-jre

WORKDIR /app

COPY target/*.jar app.jar

EXPOSE 8081

ENTRYPOINT ["java", "-jar", "app.jar"]

9. Configuración del microservicio consumidor

El consumidor fue separado del productor para implementar una arquitectura desacoplada.

9.1 Puerto de ejecución

El consumidor fue configurado en:

8082

Configuración:

`server.port=8082`

9.2 Responsabilidades

El consumidor se encarga de:

- Escuchar la cola principal de RabbitMQ.
- Validar los mensajes recibidos.
- Procesar las guías.
- Registrar resultados en Oracle.
- Registrar errores.
- Publicar o redirigir mensajes hacia la cola de errores.
- Consultar mensajes procesados.
- Exponer su propio health check.

9.3 Listener de RabbitMQ

La recepción automática de mensajes se configuró utilizando:

`@RabbitListener`

El listener permanece conectado a la cola principal y procesa cada mensaje de forma independiente.

9.4 Health check

El consumidor expone:

```
GET /api/consumidor/health
```

Esta ruta permite comprobar que el segundo microservicio se encuentra desplegado y disponible de manera independiente.

9.5 Dockerfile del consumidor

Ejemplo:

```
FROM eclipse-temurin:21-jre
```

```
WORKDIR /app
```

```
COPY target/*.jar app.jar
```

```
EXPOSE 8082
```

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

10. Configuración de Azure AD B2C

Azure AD B2C fue utilizado como proveedor de identidad para emitir tokens JWT y proteger las rutas del sistema.

10.1 Selección del tenant

Se utilizó el tenant:

```
gestionguias2026grupo13.onmicrosoft.com
```

Desde el portal de Azure se accedió a:

Azure AD B2C

10.2 Registro de la API

Se creó un registro de aplicación llamado:

Gestion_Guias_Despacho_API

Desde la sección **Información general** se obtuvo:

Id. de aplicación (cliente)

Id. de directorio (inquilino)

Estos valores permiten identificar la API y el tenant.

10.3 Exposición de la API

Dentro del registro se ingresó a:

Exponer una API

Se configuró el URI identificador de la aplicación y se definieron los permisos o roles necesarios para acceder a las operaciones protegidas.

El URI de la API se utiliza posteriormente como audiencia del token JWT.

10.4 Registro de la aplicación cliente

Para solicitar tokens mediante el flujo Client Credentials, se utilizó una aplicación cliente independiente.

Esta aplicación contiene:

- Client ID.
- Client Secret.
- Permisos de aplicación.
- Acceso autorizado a la API.

10.5 Creación del secreto

Dentro de la aplicación cliente se ingresó a:

Certificados y secretos

→ Nuevo secreto de cliente

Después de crearlo se copió la columna:

Valor

El valor fue almacenado de manera segura y no se incorporó al repositorio.

10.6 Permisos de API

Dentro de la aplicación cliente se accedió a:

Permisos de API

- Agregar un permiso
- Mis API
- Gestion_Guias_Despacho_API

Se seleccionaron los permisos requeridos y posteriormente se concedió el consentimiento administrativo.

10.7 Flujo de usuario

El flujo configurado fue:

B2C_1_Gestion_Guias_Despacho_Login

El endpoint de token utilizado tiene la siguiente estructura:

<https://gestionguias2026grupo13.b2clogin.com/>

gestionguias2026grupo13.onmicrosoft.com/

B2C_1_Gestion_Guias_Despacho_Login/

oauth2/v2.0/token

10.8 Configuración en Spring Security

Los microservicios fueron configurados como Resource Server.

Las propiedades principales corresponden a:

issuer-uri

jwk-set-uri

audience

Spring Security valida:

- Firma del token.
- Emisor.
- Audiencia.
- Fecha de expiración.
- Permisos o roles.

Las rutas de salud permanecen públicas, mientras que las rutas de negocio requieren autenticación JWT.

11. Configuración de Amazon API Gateway

Amazon API Gateway fue configurado como punto único de acceso público a los dos microservicios.

11.1 Creación de la API

Desde AWS se ingresó a:

API Gateway

→ Crear API

Se creó una API HTTP para el proyecto Gestión de Guías de Despacho.

11.2 Integraciones HTTP

Se configuraron integraciones hacia la instancia EC2.

Para el productor se utilizó:

`http://IP_PUBLICA_EC2:8081`

Para el consumidor se utilizó:

`http://IP_PUBLICA_EC2:8082`

Cada ruta fue asociada al puerto correspondiente.

11.3 Rutas del productor

Las siguientes rutas apuntan al microservicio productor:

GET /actuator/health

GET /api/productor/health

GET /api/guias

POST /api/guias

GET /api/guias/{id}

PUT /api/guias/{id}

DELETE /api/guias/{id}

GET /api/guias/descargar

POST /api/guias/subir

11.4 Rutas del consumidor

Las siguientes rutas apuntan al microservicio consumidor:

GET /api/consumidor/health

POST /api/guias/cola/consumir

GET /api/guias/cola/procesados

La ruta de estado de colas fue asociada al servicio donde se encuentra implementado su controlador:

GET /api/guias/cola/estado

11.5 Configuración del autorizador JWT

En API Gateway se creó un autorizador JWT.

Se configuraron:

Issuer

Audience

Identity source

El origen de identidad utilizado fue:

`$request.header.Authorization`

El token debe enviarse utilizando el encabezado:

Authorization: Bearer TOKEN_JWT

11.6 Asociación del autorizador

El autorizador fue asociado a las rutas protegidas.

Las rutas públicas corresponden únicamente a los health checks:

GET /actuator/health

GET /api/productor/health

GET /api/consumidor/health

Las rutas CRUD, almacenamiento y mensajería requieren JWT.

11.7 Etapa de despliegue

Se utilizó la etapa:

prod

Después de crear o modificar rutas, integraciones o autorizaciones, se implementaron los cambios en dicha etapa.

La URL base quedó con la estructura:

`https://IDENTIFICADOR.execute-api.us-east-1.amazonaws.com/prod`

12. Configuración de Docker Compose

Docker Compose fue utilizado para orquestar los tres servicios principales:

- RabbitMQ.
- Microservicio productor.
- Microservicio consumidor.

12.1 Servicio RabbitMQ

El servicio expone los puertos:

5672

15672

También incorpora un health check para verificar que RabbitMQ se encuentre disponible antes de iniciar los microservicios.

12.2 Servicio productor

El productor:

- Utiliza el puerto 8081.
- Monta el Wallet en /wallet.
- Monta EFS en /app/archivos.
- Recibe variables de Oracle, RabbitMQ, S3 y Azure.
- Depende de RabbitMQ.

12.3 Servicio consumidor

El consumidor:

- Utiliza el puerto 8082.
- Monta el Wallet en /wallet.
- Utiliza RabbitMQ.
- Se conecta a Oracle.
- Depende del estado saludable de RabbitMQ.

12.4 Ejemplo de estructura

services:

rabbitmq:

image: rabbitmq:3-management

container_name: gestion-guias-rabbitmq

ports:

- "5672:5672"

- "15672:15672"

env_file:

- .env.docker

healthcheck:

test: ["CMD", "rabbitmq-diagnostics", "ping"]

interval: 10s

timeout: 5s

retries: 10

producer:

build:

context: .

container_name: gestion-guias-producer

ports:

- "8081:8081"

env_file:

- .env.docker

volumes:

- /home/ubuntu/wallet:/wallet:ro
- /mnt/efs/guias:/app/archivos

depends_on:

rabbitmq:

condition: service_healthy

consumidor:

build:

context: ./gestionguias_consumidor

container_name: gestion-guias-consumidor

ports:

- "8082:8082"

env_file:

- .env.docker

volumes:

- /home/ubuntu/wallet:/wallet:ro

depends_on:

rabbitmq:

condition: service_healthy

12.5 Ejecución de los servicios

Para construir y levantar los contenedores se utilizó:

```
docker compose -f docker-compose.prod.yml up -d --build
```

Para comprobar el estado:

```
docker compose -f docker-compose.prod.yml ps
```

Para revisar los logs:

```
docker logs gestion-guias-productor
```

```
docker logs gestion-guias-consumidor
```

```
docker logs gestion-guias-rabbitmq
```

Para detener los servicios:

```
docker compose -f docker-compose.prod.yml down
```

13. Configuración de variables de entorno y secretos

Las credenciales fueron administradas mediante variables de entorno.

Los valores reales no se incorporaron al código fuente ni al repositorio GitHub.

Variables de Oracle

DB_URL

DB_USERNAME

DB_PASSWORD

TNS_ADMIN

Variables de RabbitMQ

RABBITMQ_HOST

RABBITMQ_PORT

RABBITMQ_USERNAME

RABBITMQ_PASSWORD

Variables de AWS S3

AWS_ACCESS_KEY_ID

AWS_SECRET_ACCESS_KEY

AWS_REGION

AWS_S3_BUCKET

Variables de Azure AD B2C

AZURE_B2C_ISSUER

AZURE_B2C_JWK_SET_URI

AZURE_B2C_AUDIENCE

Protección de los archivos

El archivo `.env.docker` fue agregado a `.gitignore`.

También se excluyeron:

Wallet

*.pem

*.p12

*.sso

.env

.env.docker

En EC2 se configuraron permisos restrictivos:

```
chmod 600 .env.docker
```

```
chmod 400 OracleCloud123.pem
```

Esta configuración evita que las credenciales sean expuestas accidentalmente.

15. Conclusiones

La configuración realizada permitió implementar una solución cloud basada en microservicios para la gestión de guías de despacho.

La separación entre productor y consumidor permitió desacoplar la recepción de solicitudes del procesamiento de mensajes. RabbitMQ proporciona la infraestructura de mensajería necesaria para administrar el flujo normal y el manejo de errores.

Oracle Autonomous Database permite mantener la información persistente mediante una conexión segura con Wallet. Amazon EFS proporciona almacenamiento compartido dentro de EC2, mientras que Amazon S3 conserva los archivos generados de forma permanente.

Azure AD B2C y Amazon API Gateway permiten proteger y centralizar el acceso a los endpoints mediante tokens JWT.

Finalmente, Docker Compose facilita el despliegue coordinado del productor, consumidor y RabbitMQ, permitiendo mantener una infraestructura organizada, reproducible y escalable.

16. Repositorio de Github

Link: https://github.com/LadyRed145/GestionGuiasDespacho_Grupo13.git

The screenshot shows the GitHub repository page for 'GestionGuiasDespacho_Grupo13' by user 'LadyRed145'. The repository is public and has 45 commits. The main branch is 'main'. The repository contains several files and folders, including .github/workflows, .mvn/wrapper, gestionguias-stack, gestionguias_consumidor, src, .dockerignore, .gitattributes, .gitignore, Dockerfile, README.md, and docker-compose.prod.yml. The right sidebar shows the 'About' section with no description, website, or topics provided. It also shows 'Releases' with no published releases and 'Packages' with no published packages. The 'Contributors' section shows 1 contributor.

File/Folder	Description	Time
.github/workflows	Refactor CI/CD pipeline for RabbitMQ and validation	6 hours ago
.mvn/wrapper	Semana 3 Gestion Guias Despacho completa	last month
gestionguias-stack	fix: update Oracle driver and producer health configuration	4 hours ago
gestionguias_consumidor	Revert "fix: add complete Oracle Wallet JDBC dependenci...	3 hours ago
src	fix: update Oracle driver and producer health configuration	4 hours ago
.dockerignore	feat: complete RabbitMQ integration with queue monitori...	last week
.gitattributes	Semana 3 Gestion Guias Despacho completa	last month
.gitignore	feat: separate producer and consumer in monorepo	8 hours ago
Dockerfile	fix: align docker wallet path and simplify container startup	3 weeks ago
README.md	Agregar README y actualizar gitignore	last month
docker-compose.prod.yml	chore: clean monorepo structure and workflow	8 hours ago



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.